Beágyazottszoftver-fejlesztés

1. gyakorlat tematikája

1. Fejlesztői kártya megismerése

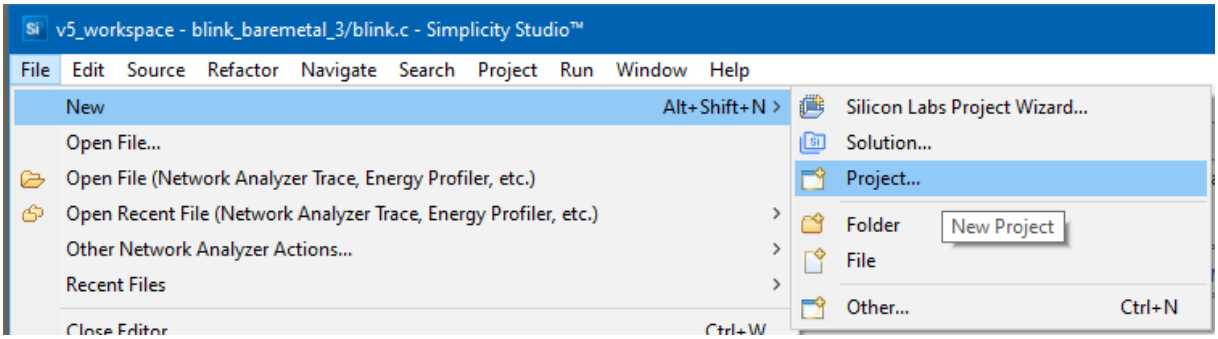
- A fejlesztői kártya rövid ismertetése, csatlakoztatása a PC-hez.
- A fejlesztői környezet indítása, megismerése.

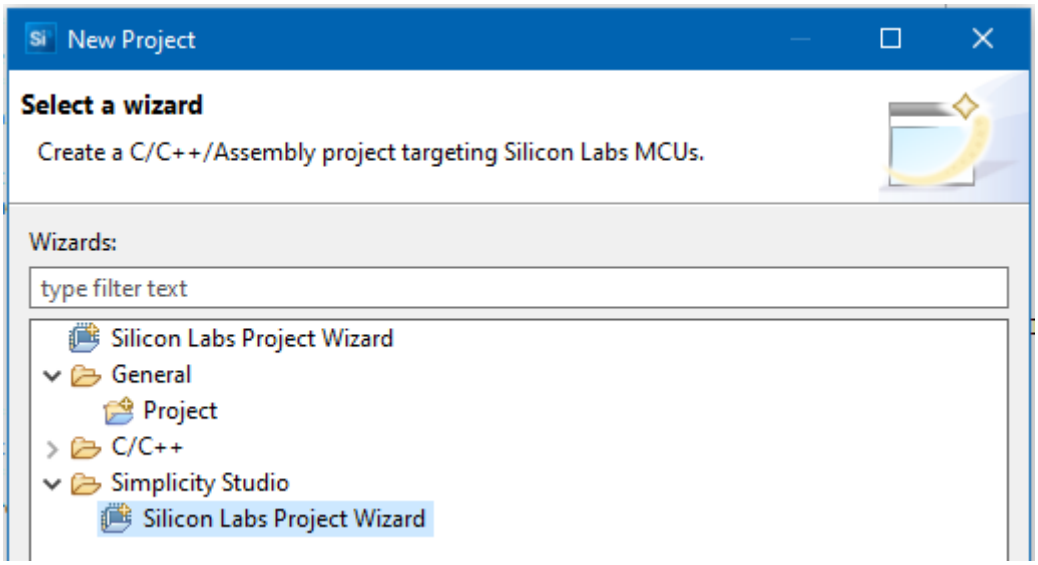
2. Fejlesztői környezet nézeteinek megismerése

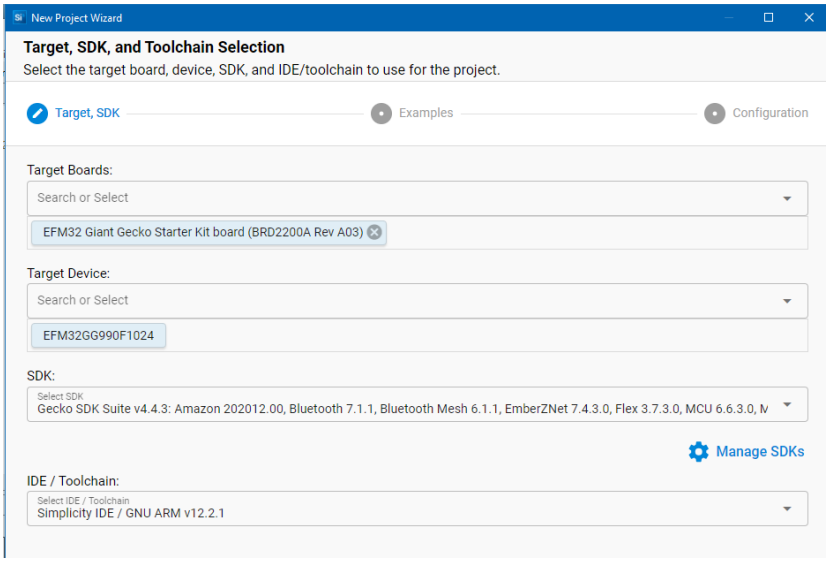
A blink projekten keresztül a fejlesztői környezet különböző nézeteinek rövid bemutatása:

- A blink (Platform – Blink Bare-metal) nevű mintaprojekt megnyitása, megismerése.
- Simplicity IDE (fejlesztés)
- Debug (program letöltése, léptetése)
- Energy profiler (áramfelvétel mérése, áramfelvétel nyomon követése függvény szinten)
    - Engedélyezés: lásd 9. fejezet.

Projekt létrehozása:







3. Mintaprogram áttekintése, API függvények bemutatása

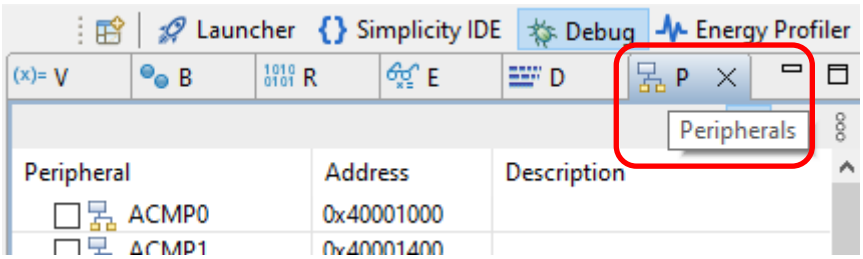
API-k (Application programming interface) szerepének rövid ismertetése. BSP (Board Support Package) rövid ismertetése.

Programstruktúra áttekintése:

- main.c, app.c és blink.c fájlok struktúrája
- init és action függvények

4. Debug lehetőségek

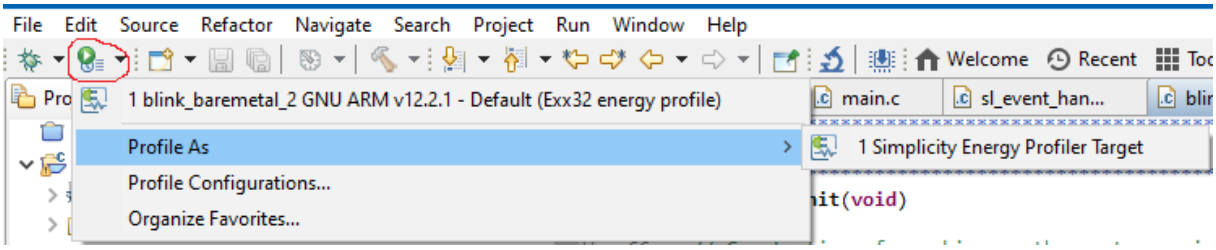
- Függvények és definíciók visszakövetése
- Változók és regiszterek értékeinek megtekintése (lokális változókat csak akkor látjuk, ha a függvényen belül vagyunk). Használjuk az Expression ablakot.
- Feladat: a blink projektben kövessük végig, hogy a LED állapotváltozása milyen függvényhívásokon keresztül történik, és milyen regiszter melyik bitjeit állítja, hogyan tükröződik a LED-ek állapota a regiszter bitjeiben. A Pheripherals ablakban lehet az adott perifériához tartozó regisztereket megnézni:



FIGYELEM! A regiszterek megtekintéséhez a periféria kiválasztását követően a Memory ablakot kell kiválasztani.

- Nézzük meg röviden a disassembly kódot. Disassembly jelentősége.

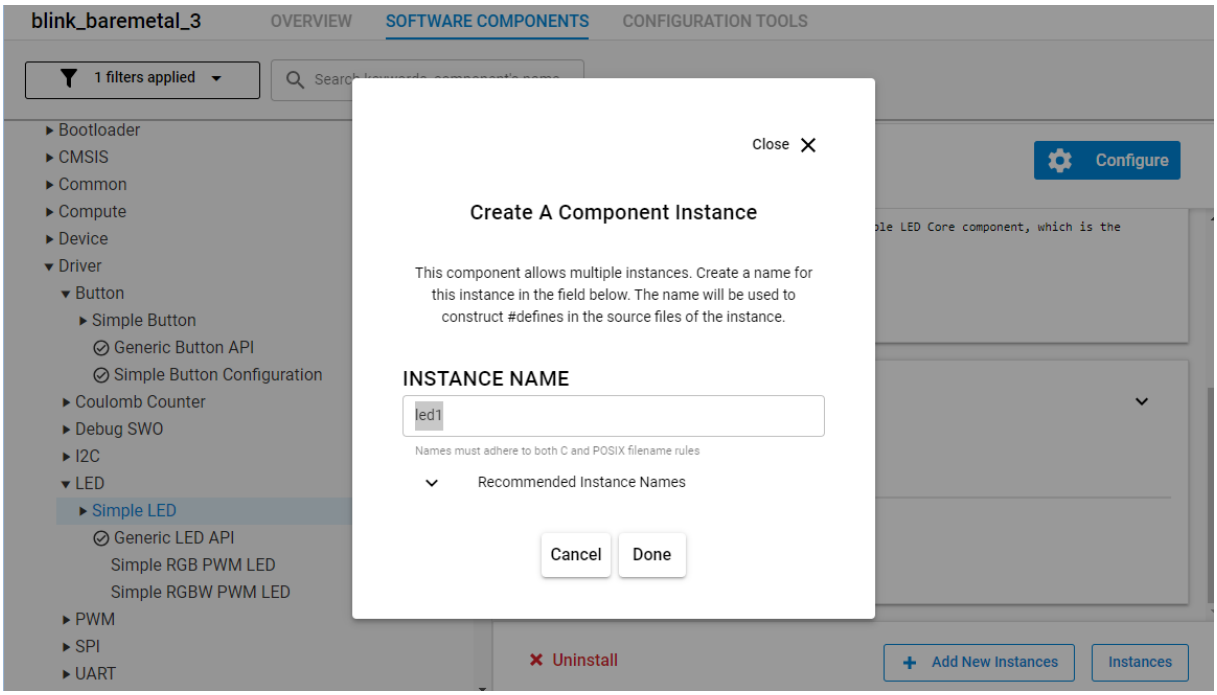
5. Energy profiling

- Energy profiler elindítása, rövid ismerkedés.
- Futtatás Energy Profiler segítségével:
- 

6. Újabb LED felvétele

Adjunk hozzá a projekthez egy újabb LED-kezelő függvény-könyvtárat:

slcp fájl → software components → Platform → Driver → LED → Simple LED → Add new instances



A blink.c fájlhoz adjuk hozzá:

```
#include "sl_simple_led.h"
#include "sl_simple_led_instances.h"
```

A LED-ek állapotának beállítása a következő függvényekkel lehetséges:

```
    sl_led_turn_on(&sl_led_led0);
    sl_led_turn_off(&sl_led_led0);
```

```
        sl_led_turn_on(&sl_led_led1);
        sl_led_turn_off(&sl_led_led1);
```

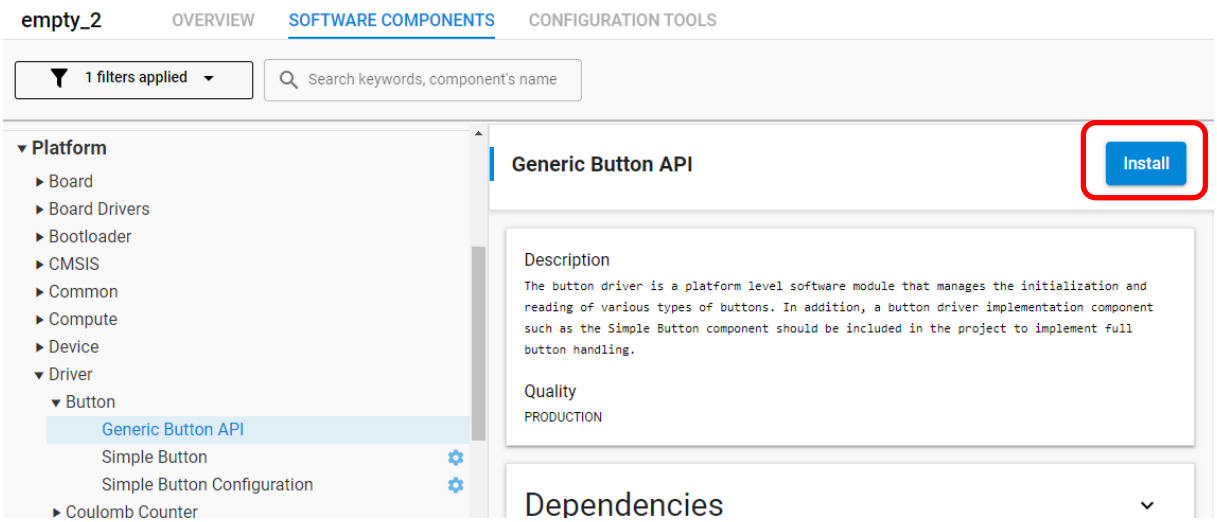
Az `sl_led_toggle` függvénnyel a LED-ek állapota váltható.

Feladatok:

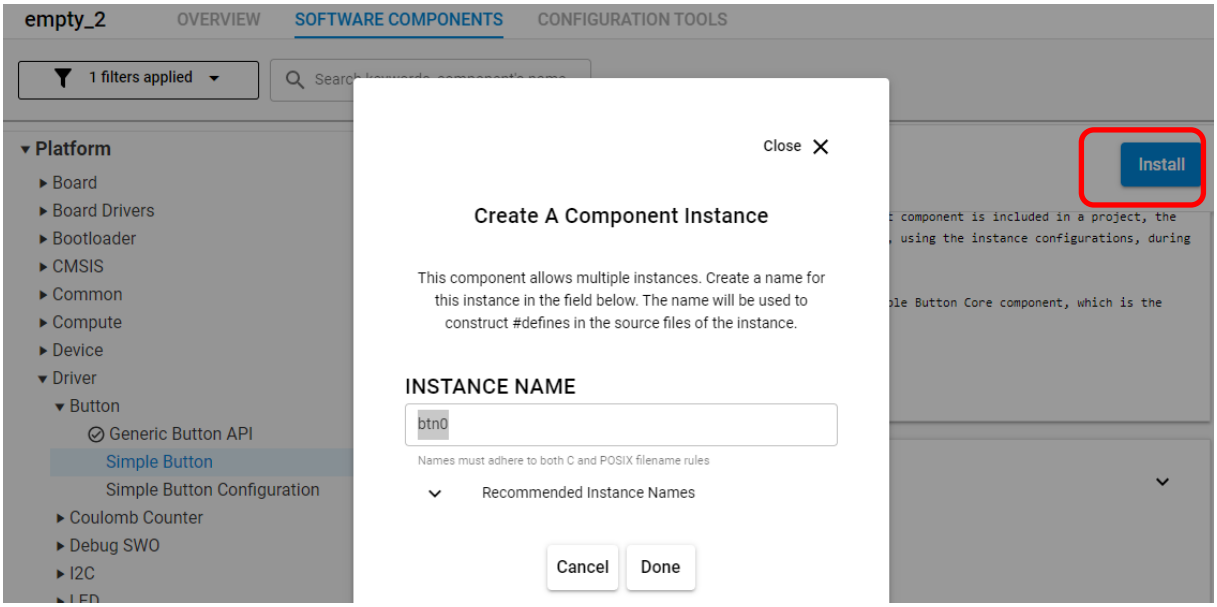
- Villogjanak a LED-ek megegyező ütemben, illetve ellenütemben.
- Duplikáljuk az időzítőt, és villogjanak a LED-ek különböző időzítéssel, például dupla periódusidővel.

7. Nyomógombok engedélyezése

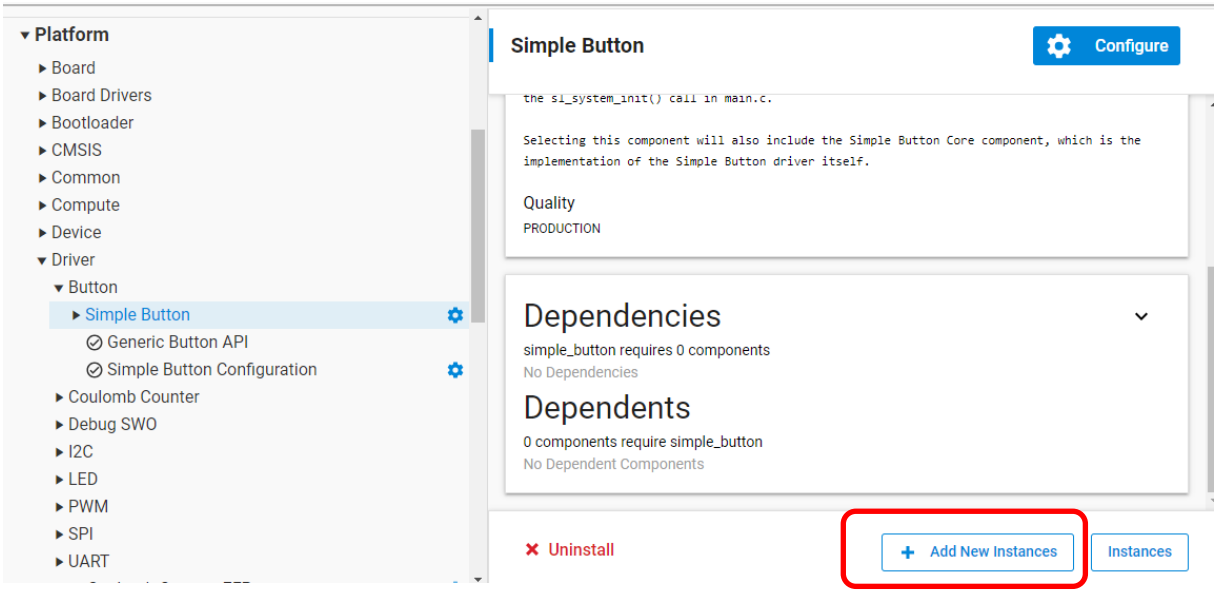
Adjuk hozzá az általános nyomógombkezelő koponenst a projekthez (Generic Button API → Install):



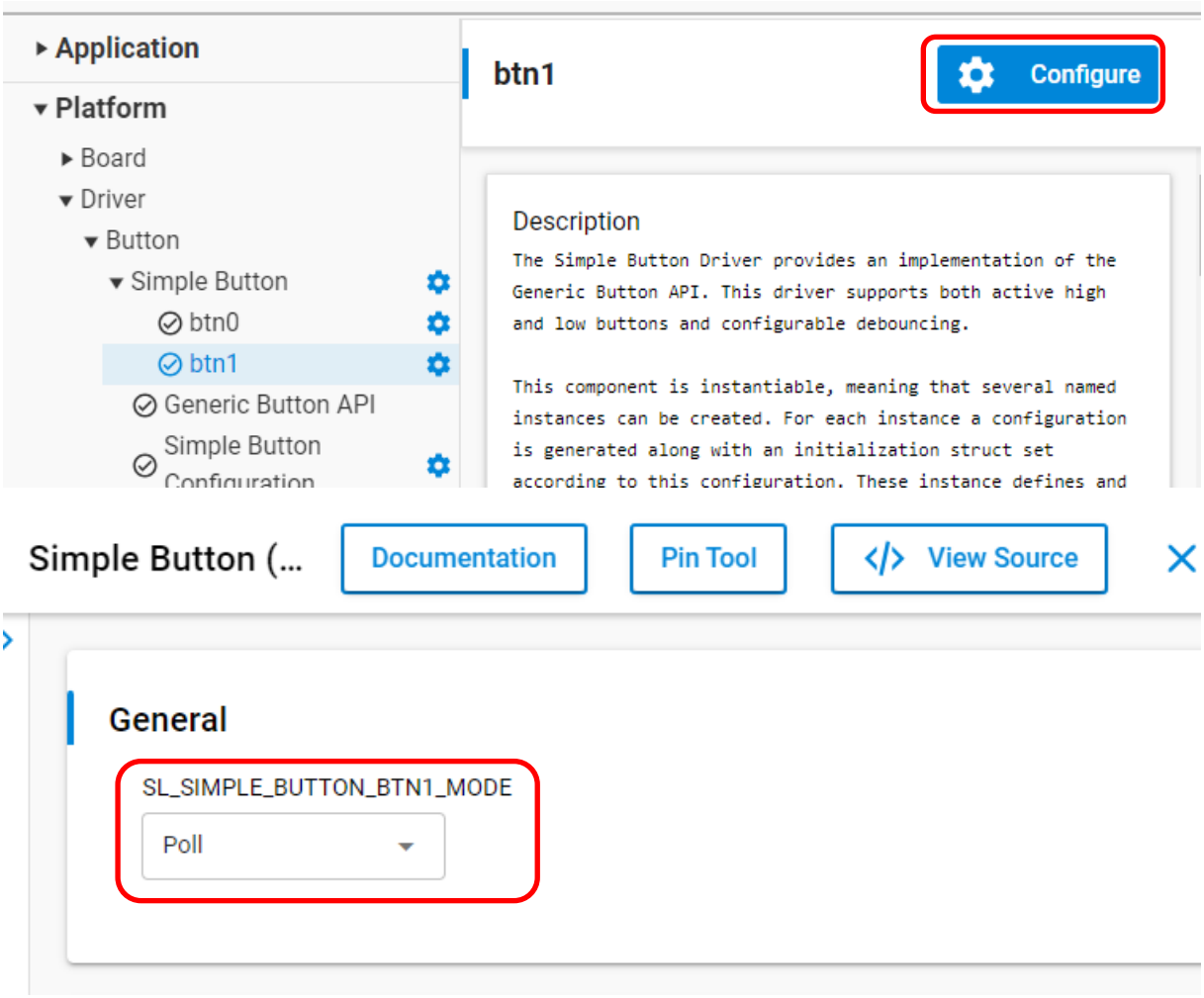
Adjuk hozzá az egyes nyomógombokat projekthez (Simple button → Install, ez a btn0-t adja hozzá):



A btn1 nyomógomb hozzáadása (Simple Button → Add New Instances):



A létrehozott nyomógombpéldányok mindegyike esetén állítsuk be, hogy lekérdezéses legyen a működésük:



A blink.c fájlhoz adjuk hozzá:

```
#include "sl_simple_button.h"
#include "sl_simple_button_instances.h"
```

A nyomógomb lekérdezése a következő módon lehetséges:

```
sl_simple_button_poll_step(&sl_button_btn0);
sl_button_state_t butt0 = sl_simple_button_get_state(&sl_button_btn0);

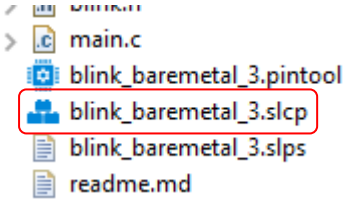
sl_simple_button_poll_step(&sl_button_btn1);
sl_button_state_t butt1 = sl_simple_button_get_state(&sl_button_btn1);
```

8. Startup kód vizsgálata

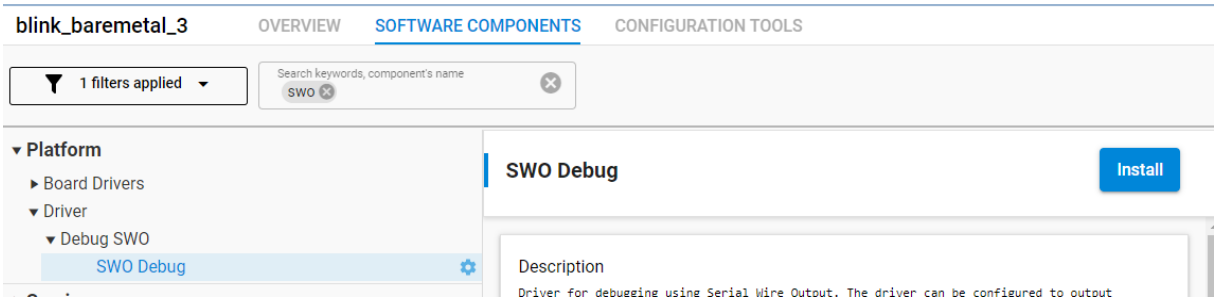
- A startup_efm32gg.c fájl bemutatása (gecgo_sdk… → platform → Device → SiliconLabs → EFM32GG → Source)
    - stack és heap definiálása
    - megszakításvektor-tábla definiálása
    - inicializált és nem inicializált változók feltöltése

9. SWO (Serial Wire Output) engedélyezése

Engedélyezzük az SWO perifériát az alábbiak szerint:



Blink_baremetal.slcp fájlra kattintunk → software components → Platform → Driver → Debug SWO → SWO Debug → Install



Konfiguráljuk az SWO Debug perifériát:

